

Operator

Kami yakin Anda tahu simbol seperti `+` atau `-` yang sering digunakan untuk melakukan operasi matematika. Ketahuilah bahwa simbol tersebut adalah sebuah **operator**. Faktanya, operator termasuk hal penting yang perlu diketahui agar Anda dapat melakukan operasi-operasi dalam membuat sebuah program. Dalam materi kali ini, kita akan membahas jenis-jenis operator yang umum digunakan pada JavaScript.

Sebelum mengenal lebih detail jenis-jenis operator yang ada, mari kita ketahui dulu dua istilah penting dalam melakukan sebuah operasi, yakni **operator** dan **operand** (operan).

- **Operator** merupakan sebuah simbol atau teks yang digunakan untuk melakukan sebuah operasi, misalnya aritmetika, penugasan, logika, tipe data, atau operasi lain yang berhubungan dengan pemrograman.
- **Operan** adalah nilai yang menjadi target dari sebuah operasi.

Sebagai contoh, ada sebuah operasi aritmetika `5 + 4`. Operatornya adalah simbol `+` serta operannya adalah `5` dan `4`. Mudah, kan? Simak ilustrasi di bawah ini agar dapat membedakannya.

Secara umum, operator dalam JavaScript terbagi menjadi tiga kelompok: **unary**, **binary**, dan **ternary**. Pengelompokan ini berdasarkan jumlah operan yang diperlukan untuk menggunakan suatu operator.

Operator aritmetika, seperti `+` termasuk dalam kelompok operator *binary* karena membutuhkan dua operan. Sementara itu, operator tipe data, seperti `typeof` adalah operator *unary* karena hanya memerlukan satu operan. Adapun operator *ternary* tentu saja membutuhkan tiga operan.

Berikut adalah contoh kode yang menunjukkan penggunaan unary, binary, dan ternary operator.

```
let age = 25;

// Unary operator
typeof age;

// Binary operator
5 + 4;
10 / 2;
age = 30;
```

```
// Ternary operator (conditional operator)
(age < 18) ? 'You are too young!' : 'Welcome onboard!';
```

Sudah jelas mengenai istilah operator dan operan beserta pengelompokan operator unary, binary, dan ternary? Jika sudah, mari kita mengenal berbagai operator yang ada dalam JavaScript.

Assignment Operator

Operator penugasan atau *assignment operator* adalah operator yang digunakan untuk memberikan nilai kepada sebuah variabel, baik inisiasi nilai baru maupun mengubah nilai yang sudah ada. Operator ini ditulis dengan simbol sama dengan (=). Operator penugasan termasuk dalam kategori *binary operator* karena membutuhkan dua operan, yaitu variabel di sisi kiri dan nilai yang akan diberikan di sisi kanan. Operator ini ditulis di antara kedua operan.

Inilah contoh penggunaan untuk inisiasi nilai.

```
// Digunakan untuk menginisiasi nilai
const name = 'Dicoding';
let location = 'Bandung';
```

Adapun berikut adalah contoh penggunaan untuk mengubah nilai.

```
// Digunakan untuk mengubah nilai
location = 'Jakarta';
```

Arithmetic Operator

Operator aritmetika adalah operator standar yang digunakan untuk proses aritmetika, seperti penambahan (+), pengurangan (-), pengalian (*), ataupun pembagian (/). Selain operasi standar yang kami sebutkan, JavaScript juga mendukung beberapa operasi aritmetik yang bisa Anda lihat pada tabel di bawah ini.

Operator	Deskripsi	Notasi	Contoh
Sisa bagi atau Modulus (%)	<i>Binary operator</i> yang mengembalikan sisa hasil bagi operan kiri dan operan kanan.	<pre>operan1 % operan2</pre>	<pre>12 % 5 mengembalikan 2. 10 % 5 mengembalikan 0.</pre>

Operator	Deskripsi	Notasi	Contoh
			$14 \% 3$ mengembalikan 2.
Increment (++)	<i>Unary operator</i> yang digunakan untuk menambahkan satu nilai terhadap operan yang diberikan. Operan harus berupa variabel yang dapat diubah nilainya. Jika operator ini digunakan sebelum operan (++x), ia akan mengembalikan nilai x yang telah ditambahkan 1. Jika operator ini digunakan setelah operan (x++), ia akan mengembalikan nilai x yang belum ditambahkan 1.	++operan; atau operan++;	Jika x adalah 3, ++x mengubah x menjadi 4 dan operasi mengembalikan 4. Jika x adalah 3, x++ operasi mengembalikan 3. Setelah itu, mengubah x menjadi 4.
Decrement (-) (-)	Sama seperti <i>increment</i> , tetapi alih-alih menambahkan, ia mengurangi nilai satu.	--operan; atau operan--;	Jika x adalah 3, --x mengubah x menjadi 2 dan operasi mengembalikan 2. Jika x adalah 3, x-- operasi mengembalikan 3. Setelah itu, mengubah x menjadi 2.
Eksponensial atau Pangkat (**)	<i>Binary operator</i> yang menghitung x (basis) dengan pangkat y (eksponen). Bentuk matematika: $x ^ y.$	operan1 ** operan2	$2 ** 3$ mengembalikan 8. $10 ** 2$ mengembalikan 100.

Berikut adalah contoh kode yang menunjukkan berbagai penggunaan operator aritmetika dalam JavaScript.

```

6 + 5; // mengembalikan 11
7 - 2; // mengembalikan 5
8 * 5; // mengembalikan 40
10 / 3; // mengembalikan 3.33

```

```
10 % 2; // mengembalikan 0
2 * (10 + 2); // mengembalikan 24
```

Catatan Tanda kurung dalam operasi aritmetika menentukan urutan operasi yang dijalankan. Operasi yang diberi tanda kurung akan dikalkulasi terlebih dulu. Hal ini juga merupakan aturan umum dalam bidang matematika.

Comparison Operator

Operator perbandingan adalah operator yang digunakan untuk membandingkan dua nilai dan mengembalikan nilai boolean (`true` atau `false`) sebagai hasil perbandingan. Salah satu penggunaan operator perbandingan adalah ketika kita berhadapan dengan skenario "Apakah x lebih besar dari y?" dan mendapatkan hasil dalam bentuk "Ya" atau "Tidak".

Berikut adalah daftar operator perbandingan yang tersedia dalam JavaScript.

Operator	Deskripsi	Notasi	Contoh
Sama (<code>==</code>)	Membandingkan dua operan, tanpa memperhatikan tipe data. Jika nilai kedua operan dianggap sama, operasi akan mengembalikan <code>true</code> , sebaliknya <code>false</code> .	<code>operan1</code> <code>==</code> <code>operan2</code>	<code>1 == 1</code> mengembalikan <code>true</code> . <code>1 == '1'</code> mengembalikan <code>true</code> . <code>2 == 6</code> mengembalikan <code>false</code> . <code>'Dicoding' == 'dicoding'</code> mengembalikan <code>false</code> .
Tidak Sama (<code>!=</code>)	Membandingkan dua operan, tanpa memperhatikan tipe data. Jika nilai kedua operan dianggap tidak sama, operasi akan mengembalikan <code>true</code> , sebaliknya <code>false</code> .	<code>operan1</code> <code>!=</code> <code>operan2</code>	<code>1 != 1</code> mengembalikan <code>false</code> . <code>1 != '1'</code> mengembalikan <code>false</code> . <code>2 != 6</code> mengembalikan <code>true</code> .

Operator	Deskripsi	Notasi	Contoh
			<pre>'Dicoding' != 'dicoding' mengembalikan true.</pre>
Identik (===)	Membandingkan dua operan beserta tipe datanya. Jika kedua operan identik, operasi akan mengembalikan true, sebaliknya false.	<pre>operan1 === operan2</pre>	<pre>1 === 1 mengembalikan true. 1 === '1' mengembalikan false. 2 === 6 mengembalikan false. 'Dicoding' === 'dicoding' mengembalikan false.</pre>
Tidak Identik (!==)	Membandingkan dua operan beserta tipe datanya. Jika kedua operan tidak identik, operasi akan mengembalikan true, sebaliknya false.	<pre>operan1 !== operan2</pre>	<pre>1 !== 1 mengembalikan false. 1 !== '1' mengembalikan true. 2 !== 6 mengembalikan true. 'Dicoding' !== 'dicoding' mengembalikan true.</pre>
Lebih dari (>)	Membandingkan dua operan, apakah operan pertama lebih dari operan kedua? Jika pernyataan benar, mengembalikan true, sebaliknya false.	<pre>operan1 > operan2</pre>	<pre>4 > 2 mengembalikan true. 4 > 5 mengembalikan false. 4 > 4</pre>

Operator	Deskripsi	Notasi	Contoh
			mengembalikan false.
Lebih dari atau Sama dengan ($\geq$)	Membandingkan dua operan, apakah operan pertama lebih dari atau sama dengan operan kedua? Jika pernyataan benar, mengembalikan true, sebaliknya false.	operan1 $\geq$ operan2	4 $\geq$ 2 mengembalikan true. 4 $\geq$ 5 mengembalikan false. 4 $\geq$ 4 mengembalikan true.
Kurang dari ($<$)	Membandingkan dua operan, apakah operan pertama kurang dari operan kedua? Jika pernyataan benar, mengembalikan true, sebaliknya false.	operan1 $<$ operan2	4 $<$ 2 mengembalikan false. 4 $<$ 5 mengembalikan true. 4 $<$ 4 mengembalikan false.
Kurang dari atau Sama dengan ($\leq$)	Membandingkan dua operan, apakah operan pertama kurang dari atau sama dengan operan kedua? Jika pernyataan benar, mengembalikan true, sebaliknya false.	operan1 $\leq$ operan2	4 $\leq$ 2 mengembalikan false. 4 $\leq$ 5 mengembalikan true. 4 $\leq$ 4 mengembalikan true.

Berikut adalah contoh kode yang menunjukkan berbagai penggunaan operator perbandingan dalam JavaScript.

```
const a = 10;
const b = 12;
```

```
console.log(a < b); // output: true
console.log(a > b); // output: false
```

Logical Operator

Operator logika digunakan untuk menetapkan logika dari dua nilai operan boolean. Dengan operator logika, kita bisa melakukan operasi gerbang logika, seperti AND dan OR. Operator logika dapat digunakan untuk menetapkan logika yang lebih kompleks.

Berikut adalah daftar operator logika yang tersedia dalam JavaScript.

Operator	Deskripsi	Notasi	Contoh
AND (&&)	<i>Binary operator</i> yang menghasilkan nilai <code>true</code> apabila kedua operan bernilai <code>true</code> (atau <i>truthy</i>). Jika salah satu operan bernilai <code>false</code> (atau <i>falsy</i>), operasi akan mengembalikan <code>false</code> .	<code>operand1</code> && <code>operand2</code>	<code>true && true</code> menghasilkan <code>true</code> . <code>false && true</code> menghasilkan <code>false</code> . <code>true && false</code> menghasilkan <code>false</code> . <code>false && false</code> menghasilkan <code>false</code> .
OR ()	<i>Binary operator</i> yang menghasilkan nilai <code>true</code> apabila salah satu operan bernilai <code>true</code> (atau <i>truthy</i>). Jika kedua operan bernilai <code>false</code> (atau <i>falsy</i>), operasi akan mengembalikan <code>false</code> .	<code>operand1</code> <code>operand2</code>	<code>true true</code> menghasilkan <code>true</code> . <code>false true</code> menghasilkan <code>true</code> . <code>true false</code> menghasilkan <code>true</code> . <code>false false</code> menghasilkan <code>false</code> .
NOT (!)	<i>Unary operator</i> yang mengubah operan bernilai <code>true</code> menjadi <code>false</code> dan <code>false</code>	<code>!operan</code>	<code>!true</code> menghasilkan

Operator	Deskripsi	Notasi	Contoh
	menjadi <code>true</code> .		<code>false</code> . <code>!false</code> menghasilkan <code>true</code> . <code>!(1 > 2)</code> menghasilkan <code>true</code> . <code>!(4 === 4)</code> menghasilkan <code>false</code> .

Berikut adalah contoh kode yang menunjukkan berbagai penggunaan operator logika dalam JavaScript.

```
// AND
console.log(true && true); // true
console.log(false && true); // false
console.log(true && false); // false
console.log(false && false); // false
console.log((5 === 5) && (3 < 5)); // true

// OR
console.log(true || true); // true
console.log(false || true); // true
console.log(true || false); // true
console.log(false || false); // false
console.log((5 === 5) || (3 > 5)); // true

// NOT
console.log(!true); // false
console.log(!false); // true
```

String Operator

Operator string adalah operator yang digunakan untuk menggabungkan dua nilai string. Operator ini menggunakan simbol `+` dan termasuk dalam kategori *binary operator*.

Berikut adalah contoh penggunaan operator string untuk menggabungkan dua nilai string.

```
const first = 'bekerja';
const second = 'sama';
```

```
const merged = first + second;  
  
console.log(merged); // Output: bekerja sama
```

Perlu diperhatikan bahwa simbol `+` memiliki fungsi ganda tergantung pada jenis operannya. Jika salah satu operan adalah string, simbol `+` akan berfungsi sebagai operator string untuk menggabungkan nilai string tersebut. Sebaliknya, jika kedua operand adalah angka, simbol `+` akan berfungsi sebagai operator aritmetika untuk melakukan penjumlahan.

Meskipun terlihat sederhana, penggunaan simbol `+` ini bisa sedikit membingungkan. Contohnya, pada operasi `"2" + 2`, hasilnya bukan `4`, melainkan `"22"`. Hal ini terjadi karena salah satu operannya adalah string sehingga simbol `+` berfungsi sebagai operator string.